

AUDIO_DSP H7 使用指南

本 PDF 说明 AUDIO_DSP 目录中 DaisySP C 移植库在 STM32H7 工程里的实际用法。库的核心模型是逐采样处理：每调用一次 Process 生成或处理一个采样点，不是一次调用生成整段音频。

1. 工程接入

- 把 AUDIO_DSP 加到 Keil 或 CubeIDE include path。
- 把需要的 aud_*.c 加入工程编译。
- 可以包含总头文件 aud_daisysp.h，也可以只包含实际模块头文件。
- 使用 float 音频样本，推荐内部范围为 -1.0 到 +1.0。
- H7 必须开启 FPU，GCC 推荐 fpv5-sp-d16 hard-float；GCC 链接时通常需要 -lm。
- 不要在音频中断里 malloc、free、printf。
- 大结构体不要放函数栈上，应定义为 static 或全局变量。

2. 音频回调基本模型

DSP 模块的状态结构体要长期保存，不能在每个采样里临时创建。典型流程是：启动时 Init，一次性设好初始参数；DMA 半缓冲或全缓冲回调里循环调用 Process；主循环或低速定时器中读取 ADC、按键、串口命令，再调用 Set 函数更新参数。

```
#include "aud_daisysp.h"
#define AUDIO_SR 48000.0f
static Aud_Osc osc;
static Aud_Adsr env;
static Aud_Svf filt;

void AudioDsp_Init(void)
{
    Aud_Osc_Init(&osc, AUDIO_SR);
    Aud_Osc_SetWaveform(&osc, AUD_WAVE_POLYBLEP_SAW);
    Aud_Osc_SetFreq(&osc, 220.0f);
    Aud_Osc_SetAmp(&osc, 0.35f);
    Aud_Adsr_Init(&env, AUDIO_SR, 1);
    Aud_Adsr_SetAttackTime(&env, 0.005f, 0.0f);
    Aud_Adsr_SetDecayTime(&env, 0.08f);
    Aud_Adsr_SetSustainLevel(&env, 0.55f);
    Aud_Adsr_SetReleaseTime(&env, 0.20f);
    Aud_Svf_Init(&filt, AUDIO_SR);
    Aud_Svf_SetFreq(&filt, 1800.0f);
    Aud_Svf_SetRes(&filt, 0.35f);
}

void AudioDsp_ProcessBlock(int16_t *out, uint32_t frames, bool gate)
{
    for (uint32_t i = 0; i < frames; ++i) {
        float e = Aud_Adsr_Process(&env, gate);
        float x = Aud_Osc_Process(&osc) * e;
```

```

        Aud_Svf_Process(&filt, x);
        x = Aud_Svf_Low(&filt);
        if (x > 0.99f) x = 0.99f;
        if (x < -0.99f) x = -0.99f;
        int16_t y = (int16_t)(x * 32767.0f);
        out[2 * i + 0] = y;
        out[2 * i + 1] = y;
    }
}

```

如果输出到 12-bit DAC，把 float 转成 $2048 + x * 2047$ ；如果输出到 I2S codec，通常转成 int16_t 或 int32_t 双声道交错数据。

3. 合成器不是单个接口

合成器通常由多个模块串起来：音源、音高控制、包络、滤波、效果、限幅。比如按键控制持续音时，按下时 `gate=true`，松开时 `gate=false`；音频回调里每个采样调用 ADSR 得到当前音量，再乘到 `oscillator` 输出上。

音源可选 `aud_osc`、`aud_fm2`、`aud_blosc`、`aud_variableshapeosc`、`aud_harmonic_osc`。包络可选 `aud_adsr` 或 `aud_adenv`。滤波可选 `aud_svf`、`aud_moogladder`、`aud_tone`。效果可选 `chorus`、`flanger`、`phaser`、`reverb`。最后输出前要限制幅度，避免爆音。

```

static bool key_gate;

void Synth_NoteOn(float freq)
{
    Aud_Osc_SetFreq(&osc, freq);
    key_gate = true;
}

void Synth_NoteOff(void)
{
    key_gate = false;
}

/* 在音频采样循环中 */
float e = Aud_Adsr_Process(&env, key_gate);
float x = Aud_Osc_Process(&osc) * e;
Aud_Svf_Process(&filt, x);
x = Aud_Svf_Low(&filt);

```

4. 鼓和一次性触发模块

鼓模块不是 `gate` 持续音，而是 `trigger` 触发一次事件。可以用按键上升沿、定时器、节拍器 `aud_metro` 触发。

```

static Aud_Metro metro;
static Aud_SyntheticBassDrum kick;

void Drum_Init(void)
{
    Aud_Metro_Init(&metro, AUDIO_SR, 2.0f);
    Aud_SyntheticBassDrum_Init(&kick, AUDIO_SR);
}

```

```

    Aud_SyntheticBassDrum_SetFreq(&kick, 55.0f);
    Aud_SyntheticBassDrum_SetDecay(&kick, 0.6f);
}

float Drum_Process(void)
{
    bool trig = Aud_Metro_Process(&metro) != 0;
    return Aud_SyntheticBassDrum_Process(&kick, trig);
}

```

5. 参数更新原则

- 音频回调里只做必须的逐采样计算。
- ADC、按键、旋钮、串口解析放在主循环或低速定时器。
- 参数变化时才调用 SetFreq、SetRes、SetTime 这类函数。
- 连续变化参数建议用 aud_port、aud_line 或一阶低通平滑。
- 多个音源混音时要先降幅，比如 4 个 voice 每个 0.20。
- 频率不要超过采样率一半，振荡器和滤波器都要留余量。

6. 大内存模块

aud_delayline 需要调用者提供 float buffer; aud_chorus 内置双声道延迟线; aud_flanger 内置延迟线; aud_pitchshifter 内置 4096 float 延迟线; aud_reverb32 约 98936 float, 很大, 不能放栈上; aud_looper 需要调用者提供录音 buffer; aud_pluck 需要调用者提供 Karplus buffer; aud_polypluck 内置多 voice buffer; aud_comb 和 aud_allpass 内置固定 buffer。

建议所有大模块都定义为 static/global, 必要时放 AXI SRAM 或 DTCM, 并检查 linker script 或 scatter file。

7. 模块索引

核心和工具: aud_daisysp 是总入口, aud_dsp 提供常量、clamp、soft clip、BLEP 和映射函数, aud_delayline 是通用分数延迟线, aud_dcblock 去直流, aud_metro 产生周期 trigger, aud_line 生成线性斜坡, aud_port 平滑参数, aud_maytrig 概率触发, aud_samplehold 采样保持。

控制包络: aud_adsr 用 gate 工作, 适合键盘和持续音; aud_adenv 用 trigger 工作, 适合鼓、pluck、滤波扫频; aud_phasor 输出 0 到 1 相位; aud_jitter 和 aud_smooth_random 产生随机控制源。

振荡器和合成音源: aud_osc 是基础多波形振荡器; aud_blosc 是带限 saw/square/triangle; aud_fm2 是双算子 FM; aud_formantosc 是共振峰振荡器; aud_harmonic_osc 可设置 16 个谐波幅度; aud_oscillatorbank 是谐波 bank; aud_variablesawosc、aud_variableshapeosc、aud_vosim、aud_zoscillator 提供更多音色。

噪声和粒子: aud_whitenoise、aud_clockednoise、aud_dust、aud_fractal_noise、aud_grainlet、aud_particle。

滤波器: aud_svf 同时输出 low、high、band、notch、peak; aud_onepole 是一阶滤波; aud_tone 是 tone 低通/高通; aud_mode 是模态滤波; aud_moogladder 是 Moog ladder 风格低通; aud_soap、aud_nlfilt、aud_allpass、aud_comb 是更特殊的滤波和混响构件。

动态和混音: `aud_compressor`、`aud_limiter`、`aud_crossfade`、`aud_balance`。

效果: `aud_autowah`、`aud_bitcrush`、`aud_decimator`、`aud_sampleratereducer`、`aud_chorus`、`aud_flanger`、`aud_phaser`、`aud_tremolo`、`aud_overdrive`、`aud_wavefolder`、`aud_fold`、`aud_pitchshifter`、`aud_reverb`。

物理建模和弦: `aud_karplusstring`、`aud_pluck`、`aud_polypluck`、`aud_modalvoice`、`aud_stringvoice`、`aud_resonator`、`aud_drip`。

鼓合成: `aud_analogbassdrum`、`aud_analogsnaredrum`、`aud_synthbassdrum`、`aud_synthsnaresdrum`、`aud_hihat`。

采样和循环: `aud_granularplayer` 播放已有 sample buffer, `aud_looper` 做录放循环, 需要外部录音 buffer。

8. 本次检查已经修正的问题

- `aud_daisysp.h` 原来包含已删除的 `aud_oscbank.h`, 已经改为 `aud_oscillatorbank.h`。
- `aud_delayline.h` 的 `Aud_DelayLine_Allpass` 原来是占位返回 0.0f, 已经按 DaisySP 逻辑实现为真正的 allpass 读写步骤。
- `aud_harmonic_osc` 原来是 TODO 空实现, 已经补齐 DaisySP `HarmonicOscillator` 的 C 版实现, 并保留旧的 `Aud_HarmonicOscillator` 兼容别名。
- 已用 STM32CubeCLT 的 `arm-none-eabi-gcc` 对 `AUDIO_DSP` 下所有 .c 做语法检查。

9. 上板调试顺序

先只输出 `aud_osc` 正弦波, 确认 DAC/I2S、DMA、采样率和声道格式。然后加 `aud_adsr` 验证按键 `gate`。再加 `aud_svf` 或 `aud_tone`。然后再加 `chorus`、`flanger`、`reverb` 这类大 buffer 效果。最后验证复杂物理模型、粒化、`looper`。

每一步都把最终输出限制在 -0.9 到 +0.9, 用示波器、耳机小音量或串口统计确认没有 NaN、Inf 和削顶爆音。